

The cProfile Profiler

Although the `timeit` module is useful for measuring small code snippets, the `cProfile` module is more effective for analyzing entire functions or programs. *Profiling* analyzes your program's speed, memory usage, and other aspects systematically. The `cProfile` module is Python's *profiler*, or software that can measure a program's runtime as well as build a profile of runtimes for the program's individual function calls. This information provides substantially more granular measurements of your code.

To use the `cProfile` profiler, pass a string of the code you want to measure to `cProfile.run()`. Let's look at how `cProfile` measures and reports the execution of a short function that sums all the numbers from 1 to 1,000,000:

```
import time, cProfile
def addUpNumbers():
    total = 0
    for i in range(1, 1000001):
        total += i

cProfile.run('addUpNumbers()')
```

When you run this program, the output will look something like this:

```
4 function calls in 0.064 seconds

Ordered by: standard name

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.000    0.000    0.064    0.064 <string>:1(<module>)
      1   0.064    0.064    0.064    0.064 test1.py:2(addUpNumbers)
      1   0.000    0.000    0.064    0.064 {built-in method builtins.exec}
      1   0.000    0.000    0.000    0.000 {method 'disable' of
'_lsprof.Profiler' objects}
```

Each line represents a different function and the amount of time spent in that function. The columns in `cProfile.run()`'s output are:

ncalls The number of calls made to the function